



PRD - Programa de Combinatoria: Clave Criptográfica (Tema 6)

1. Resumen Ejecutivo

Desarrollo de un programa en Python que resuelva problemas de combinatoria, específicamente para calcular la cantidad de claves posibles de longitud n usando m símbolos disponibles con repetición permitida. El programa debe ser educativo, validar entradas y presentar resultados de manera clara.

2. Visión del Producto

Crear una herramienta que permita a estudiantes entender y resolver problemas de conteo combinatorio, combinando rigor matemático con una experiencia de usuario intuitiva.

3. Objetivos

- ☒ Implementar un algoritmo correcto para calcular variaciones con repetición
 - ☒ Validar entradas de usuario de forma robusta
 - ☒ Presentar resultados de manera legible y educativa
 - ☒ Documentar el proceso matemático y de desarrollo
 - ☒ Proporcionar pruebas unitarias que validen la solución
-

4. Descripción del Problema

Enunciado

Se quiere crear una clave de longitud **n** usando **m** símbolos disponibles. Los símbolos pueden repetirse y el orden importa.

Ejemplo

Claves de 4 caracteres usando 10 dígitos (0-9) → **10,000 combinaciones posibles**

Tipo de Conteo

Variación con Repetición (VR)

- Fórmula: $VR(m, n) = m^n$
 - m = cantidad de símbolos disponibles
 - n = longitud de la clave
-

5. Requisitos Funcionales

RF-1: Función Principal

- **Nombre:** `resolver(m, n)`
- **Entrada:**
 - `m` (int): cantidad de símbolos disponibles
 - `n` (int): longitud de la clave
- **Salida:** (int) cantidad de claves posibles
- **Fórmula:** m^n

RF-2: Interfaz de Usuario

- El programa debe ejecutarse desde consola
- Explicar brevemente qué problema resuelve
- Solicitar datos necesarios al usuario
- Mostrar resultado en formato legible: *"La cantidad de claves posibles es: XXXX"*

RF-3: Validación de Entrada

-  Rechazar números negativos

-  Rechazar letras cuando se esperan números
-  Rechazar valores inválidos (ej: $m=0$, $n=0$)
-  Mostrar mensajes de error claros y comprensibles
-  Permitir reintentos después de error

RF-4: Cálculo Correcto

- Calcular m^n correctamente
 - Devolver resultado como número entero
 - Manejar casos límite ($m=1$, $n=1$, valores grandes)
-

6. Requisitos No Funcionales

RNF-1: Usabilidad

- Interfaz clara y directa
- Mensajes comprensibles en español
- Guía paso a paso al usuario

RNF-2: Rendimiento

- Respuesta inmediata (< 1 segundo)
- Manejo correcto de números grandes

RNF-3: Robustez

- Captura de excepciones
- Validación exhaustiva de entrada
- No debe "crashear" con entradas inesperadas

RNF-4: Mantenibilidad

- Código limpio y bien estructurado
 - Comentarios explicativos
 - Funciones reutilizables
-

7. Estructura del Proyecto

```
tp_combinatoria/  
├── tp_combinatoria.py          # Programa principal  
├── informe.pdf o informe.docx # Documentación  
├── pruebas.txt                # Lote de pruebas  
└── README.md                  # Instrucciones de uso
```

8. Especificación de la Función

Prototipo

```
def resolver(m, n):  
    """  
    Calcula la cantidad de claves posibles.  
  
    Args:  
        m (int): Cantidad de símbolos disponibles  
        n (int): Longitud de la clave  
  
    Returns:  
        int: Cantidad de variaciones con repetición ( $m^n$ )  
  
    Raises:  
        ValueError: Si  $m < 1$  o  $n < 1$   
        TypeError: Si los argumentos no son enteros  
    """
```

Casos de Uso

Entrada (m, n)	Resultado Esperado	Descripción
(10, 4)	10,000	Ejemplo del enunciado
(2, 3)	8	Claves binarias de 3 bits
(26, 5)	11,881,376	Contraseñas con letras
(1, 5)	1	Un solo símbolo

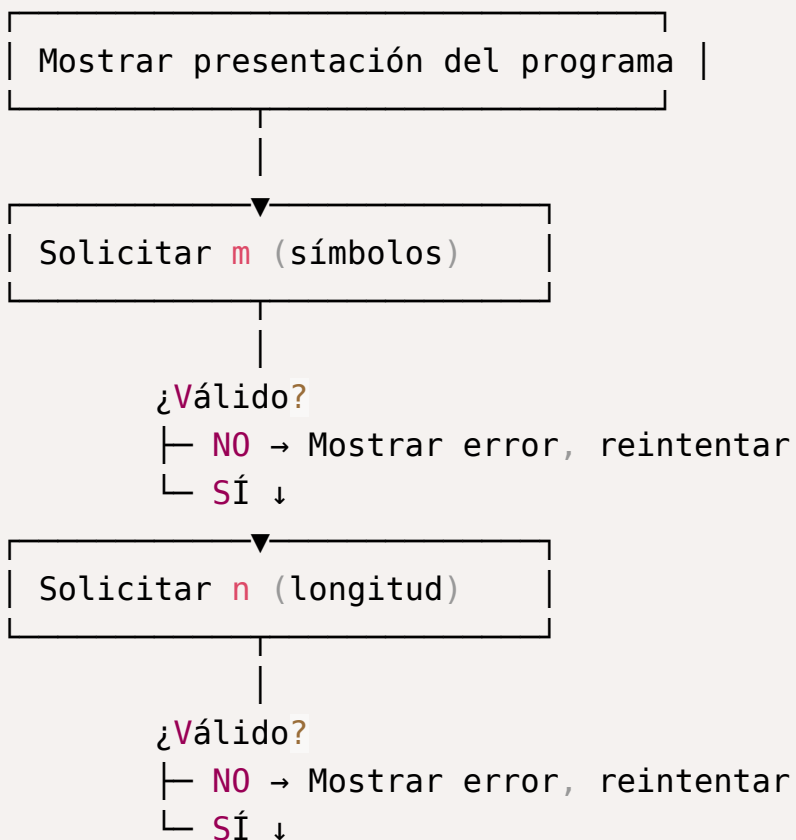
Entrada (m, n)	Resultado Esperado	Descripción
(10, 1)	10	Claves de 1 dígito

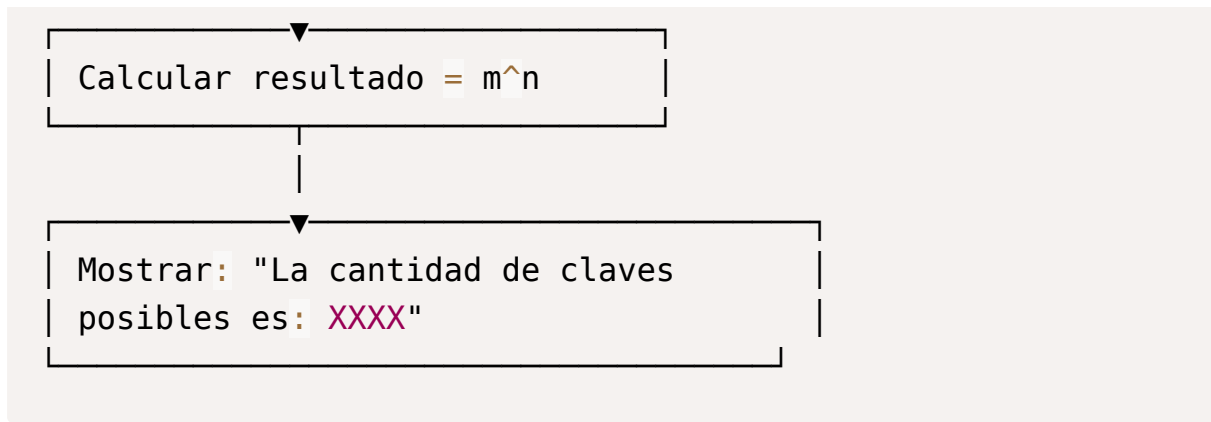
9. Validación de Entrada

Reglas

- ✓ $m \geq 1$ (al menos 1 símbolo)
- ✓ $n \geq 1$ (longitud mínima de 1)
- ✓ Ambos valores deben ser enteros
- ✗ $m < 1 \rightarrow$ Error: "El valor de m debe ser un número natural (entero positivo)"
- ✗ $n < 1 \rightarrow$ Error: "La longitud de la clave debe ser como mínimo 1"
- ✗ Letras $\rightarrow$ Error: "Por favor, ingrese un número válido"

10. Flujo de Ejecución





11. Lote de Pruebas Requeridas

Mínimo 3 pruebas con:

- Entrada (m, n)
- Resultado esperado
- Resultado obtenido
- Estado (✓ PASÓ / ✗ FALLÓ)

Ejemplo:

PRUEBA 1: Caso ejemplo del enunciado

Entrada: m=10, n=4

Esperado: 10000

Obtenido: 10000

Estado: ✓ PASÓ

PRUEBA 2: Caso trivial

Entrada: m=2, n=3

Esperado: 8

Obtenido: 8

Estado: ✓ PASÓ

PRUEBA 3: Valor grande

Entrada: m=26, n=5

Esperado: 11881376

Obtenido: 11881376

Estado: ✓ PASÓ

12. Informe Requerido (PDF o DOCX)

El informe debe incluir:

1. Datos de identificación

- Nombre de integrantes
- Tema asignado (Tema 6)

2. Explicación del problema

- Con propias palabras
- Contexto y aplicaciones

3. Identificación matemática

- Tipo: Variación con repetición
- Justificación

4. Fórmula

- $VR(m, n) = m^n$
- Deducción breve

5. Algoritmo implementado

- Pseudocódigo o explicación
- Decisiones de diseño

6. Ejemplos de ejecución

- Capturas de pantalla del programa
- Al menos 2 ejemplos

7. Lote de pruebas

- 3 casos con entrada/salida/resultado

8. Declaración de uso de IA (si aplica)

- Qué herramienta
 - Qué preguntas
 - Qué aceptaron/modificaron
 - Decisiones propias
-

13. Criterios de Aceptación

Criterio	Descripción	Estado
Función resolver()	Calcula correctamente m^n	☐
Validación de m	Rechaza $m < 1$	☐
Validación de n	Rechaza $n < 0$	☐
Validación de tipo	Rechaza no-enteros	☐
Interfaz	Muestra instrucciones claras	☐
Resultado legible	Presenta frase comprensible	☐
Manejo de errores	Mensajes claros y precisos	☐
Pruebas	Mínimo 3 casos funcionan	☐
Informe	Completo y bien documentado	☐

14. Rúbrica de Evaluación

Nota máxima: 10 puntos

1. Funcionamiento (6 pts)

- Identifica tipo de conteo: **1 pt**
- Fórmula correcta: **1.5 pts**
- Función resolver() correcta: **1.5 pts**
- Número entero correcto: **1 pt**
- Casos límite: **1 pt**

2. Informe (2 pts)

- Explicación clara: **0.4 pts**
- Justificación fórmula: **0.6 pts**
- Explicación algoritmo: **0.4 pts**
- Ejemplos/capturas: **0.3 pts**
- Declaración IA: **0.3 pts**

3. UX y Validación (1 pt)

- Instrucciones claras: **0.25 pts**

- Validación de entrada: **0.35 pts**
- Mensajes error: **0.2 pts**
- Resultado legible: **0.2 pts**

4. Lote de Pruebas (1 pt)

- Mínimo 3 pruebas: **1 pt**
-

15. Notas de Implementación

Consideraciones Técnicas

- Usar `**` para exponenciación en Python
- Validar tipos con `isinstance()`
- Usar excepciones (`try/except`) para robustez
- Implementar función `input_validado()` para reutilización

Casos Límite

- $m=1, n=100 \rightarrow \text{Resultado} = 1$
 - $m=100, n=1 \rightarrow \text{Resultado} = 100$
 - $m=0 \rightarrow \text{Error (sin símbolos)}$
 - $n=0 \rightarrow \text{Error (la longitud debe ser como mínimo 1)}$
-

16. Entregables Finales

- ✓ `tp_combinatoria.py` -- Programa funcional
 - ✓ `informe.pdf` o `informe.docx` -- Documentación completa
 - ✓ `pruebas.txt` -- Lote de 3+ pruebas documentadas
 - ✓ `README.md` -- Instrucciones de ejecución (opcional)
-

Versión: 1.0

Fecha: Mayo 2026

Asignatura: Matemática Discreta - Segundo Parcial